ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



TÀI LIỆU HỌC
THIẾT KẾ PHẦN MỀM

KIẾN THỨC NỀN PHASE 3

KIẾN TRÚC, UML VÀ THIẾT KẾ PHƯƠNG THỨC

Mã môn học: CO3001

Giảng viên hướng dẫn: Cô Trần Thị Ngọc Trâm

Mã lớp: L02

Nhóm: K24 Warriors

Học kỳ: Học kỳ 261

TP. Hồ Chí Minh, tháng 10/2026

Mục lục

I Từ phân tích yêu cầu đến thiết kế kiến trúc **3**
1 Phase 3 cần trả lời điều gì? . . . 3
2 Bốn mức nhìn một phần mềm . . . 3
3 Architecture, view và diagram . . . 3
4 Các khái niệm dễ nhầm . . . 4
5 Layer, tier và module nghiệp vụ . . . 4
6 Một số lựa chọn kiến trúc . . . 5
7 Đánh giá một quyết định thiết kế . . . 5

II Deployment View **7**
1 Lý thuyết từ đầu . . . 7
2 Node và device . . . 7
3 Execution environment . . . 7
4 Artifact và quan hệ triển khai . . . 8
5 Communication path và protocol . . . 8
6 System boundary và hệ thống ngoài . . . 9
7 Local, Docker và production . . . 9
8 Cách tự dựng một Deployment View . . . 9

III Development/Implementation View **10**
1 Lý thuyết từ đầu . . . 10
2 Package và component . . . 10
3 Provided interface và required interface . . . 10
4 Port ở biên component . . . 11
5 Dependency và chiều mũi tên . . . 11
6 Bốn vai trò thường gặp trong backend . . . 11
7 Dependency inversion: vì sao cần interface? . . . 12
8 Repository, service, DTO và adapter . . . 12
9 Từ view đến cấu trúc thư mục . . . 13
10 Cross-cutting concerns . . . 13

IV Class diagram: lý thuyết và từng ký hiệu **14**
1 Class diagram dùng để làm gì? . . . 14
2 Class và object . . . 14
3 Hộp class và ba ngăn . . . 14
4 Attribute: tên, kiểu, số lượng và giá trị mặc định . . . 15
5 Visibility: ai được truy cập? . . . 15
6 Operation và signature . . . 15
7 Static, abstract và interface . . . 16
8 Stereotype, enumeration và constraint . . . 16
9 Association: các đối tượng có liên hệ . . . 17
10 Multiplicity: đọc số ở đầu nào? . . . 17
11 Navigability: biết cách đi tới đối tượng khác . . . 18
12 Aggregation: quan hệ toàn thể–bộ phận yếu . . . 18
13 Composition: sở hữu bộ phận chặt hơn . . . 18
14 Generalization: kế thừa kiểu . . . 19
15 Realization: hiện thực hợp đồng . . . 19
16 Dependency: cần dùng một kiểu khác . . . 19
17 Association class: dữ liệu thuộc về một quan hệ . . . 20
18 Chọn class từ đâu và tránh vẽ quá nhiều . . . 20

V Thiết kế lớp và mô tả phương thức **21**
1 Vì sao tên method chưa đủ? . . . 21
2 Mô tả class trước khi mô tả operation . . . 21
3 Những trường cần có trong một hợp đồng . . . 21
4 Precondition, validation, guard và invariant . . . 22
5 Return value và lỗi . . . 22
6 Query và command . . . 23
7 Transaction và thay đổi nguyên tử . . . 23
8 Ba cơ chế cạnh tranh cần hiểu . . . 23
9 Idempotency: thử lại mà không tạo tác động trùng . . . 24
10 Thời gian, tác vụ nền và dữ liệu ngoài . . . 24
11 Từ sequence tới method descriptions . . . 24

VI Thiết kế test case (Bonus) **25**
1 Test case là gì? . . . 25
2 Các mức kiểm thử . . . 25
3 Stub, mock và fake . . . 25
4 Cấu trúc một test case . . . 26
5 Equivalence partition và boundary value . . . 26
6 Decision table và state-transition test . . . 27
7 Kiểm thử cạnh tranh và lỗi giữa chừng . . . 27
8 Kiểm thử NFR . . . 27
9 Ví dụ test đã thiết kế, chưa phải kết quả chạy . . . 28

VII Ví dụ hoàn chỉnh: đăng ký khóa học **29**
1 Bối cảnh và quy tắc của ví dụ . . . 29
2 Deployment View của ví dụ . . . 30
3 Development/Implementation View của ví dụ . . . 31
4 Class diagram miền . . . 32
5 Class diagram điều phối và lưu trữ . . . 33
6 Hợp đồng các operation miền . . . 35
7 Hợp đồng điều phối và policy . . . 35
8 Hợp đồng của interface lưu trữ . . . 36
9 Hợp đồng implementation và bao phủ toàn bộ method . . . 36
10 Đọc đường xử lý của EnrollmentService . . . 37
11 Hai request cạnh tranh một chỗ . . . 38
12 Test case rút ra trực tiếp từ thiết kế . . . 38

VIII Áp dụng cho SEMH và tự luyện **39**
1 Đầu vào hiện hành của nhóm . . . 39
2 Nhìn hệ thống bằng hai view kiến trúc . . . 39
3 Gợi ý trách nhiệm theo phân hệ . . . 39
4 Lifeline của Phase 2 chuyển sang thiết kế như thế nào? . . . 40
5 Ví dụ suy nghĩ thiết kế cho UC-RES-01 . . . 41
6 IoT giả lập vẫn cần được thiết kế đầy đủ . . . 41
7 Thiết kế phân hệ mô phỏng (SUB-SIM) . . . 42
8 Cách chia việc mà không nhân bản thiết kế . . . 43
9 Bài tập tự luyện và gợi ý đáp án . . . 43
10 Tài liệu đối chiếu . . . 44

I Từ phân tích yêu cầu đến thiết kế kiến trúc

1 Phase 3 cần trả lời điều gì?

Phase 1 xác định hệ thống cần làm gì và điều kiện nghiệp vụ nào phải được giữ. Phase 2 làm rõ người dùng thao tác ra sao, các bước xử lý và các thành phần trao đổi với nhau. Phase 3 quyết định **phần mềm được tổ chức và hiện thực bằng những thành phần nào**, chúng chạy ở đâu và hợp tác qua hợp đồng nào.

Ví dụ: "Không được xác nhận hai lượt đặt cho cùng một xe" là yêu cầu. "Hai yêu cầu phải cạnh tranh trong một transaction có ràng buộc duy nhất" là quyết định thiết kế để giữ yêu cầu đó. Viết lại requirement bằng từ khó hơn chưa phải là thiết kế.

Theo đề bài, Submission 3 gồm **Deployment View**, **Development / Implementation View**, **Class Diagram và mô tả mọi phương thức xuất hiện trên sơ đồ**. Test case là phần bonus. Không có yêu cầu phải dùng microservices, cloud, ngôn ngữ cụ thể hoặc database thật. Tuy nhiên, nhóm vẫn phải giải thích thiết kế bảo đảm các quy tắc đã cam kết như thế nào.

2 Bốn mức nhìn một phần mềm

Mức nhìn	Câu hỏi	Ví dụ
Nghiệp vụ	Người dùng muốn đạt mục tiêu gì?	Đăng ký khóa học
Kiến trúc	Chia hệ thống thành những khối trách nhiệm nào?	Web, API, miền đăng ký, lưu trữ
Thiết kế lớp	Đối tượng nào giữ dữ liệu, hành vi và quan hệ?	Course, Registration, EnrollmentService
Phương thức	Một thao tác nhận gì, kiểm tra gì và thay đổi gì?	enroll(request)

Không phải bốn sản phẩm độc lập. Chúng phải diễn tả cùng một hệ thống ở các mức chi tiết khác nhau. Một class không nhất thiết ứng với một màn hình hoặc một bảng dữ liệu.

3 Architecture, view và diagram

Architecture: kiến trúc

Kiến trúc là các quyết định lớn về trách nhiệm, cách chia phần mềm, phụ thuộc, dữ liệu, tích hợp và các ràng buộc khó thay đổi. Tên công nghệ chỉ là một phần. Liệt kê React, Node.js và PostgreSQL mà không nêu chúng liên hệ thế nào thì chưa giải thích được kiến trúc.

View: một góc nhìn phục vụ một mối quan tâm

Deployment View nhìn hệ thống khi chạy: máy, môi trường chạy và kết nối. Development/Implementation View nhìn mã nguồn: module, package, component và chiều phụ thuộc. Class diagram nhìn cấu trúc các kiểu đối tượng. Một view có thể cần hình cùng phần giải thích, không chỉ một ảnh.

Diagram: cách biểu diễn một phần của view

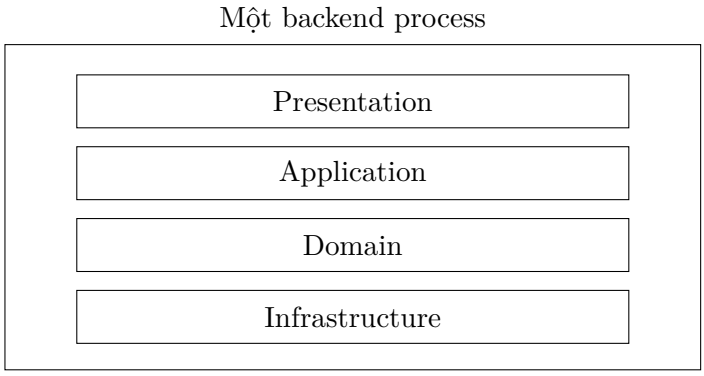
Không phải hình nào có các ô và mũi tên cũng là cùng một loại sơ đồ. Cần nói rõ ô là máy, module hay class; mũi tên là kết nối mạng, phụ thuộc mã nguồn hay quan hệ đối tượng. Nếu không nêu ý nghĩa, người đọc rất dễ suy diễn sai.

4 Các khái niệm dễ nhầm

Khái niệm	Cách hiểu
Module/package	Cách gom mã nguồn theo trách nhiệm. Có thể chỉ là thư mục hoặc package trong cùng ứng dụng.
Component	Một thành phần có trách nhiệm và hợp đồng tương tác rõ. Không mặc nhiên là một server riêng.
Process	Một lần chương trình đang thực thi, có bộ nhớ và trạng thái chạy. Một process có thể chứa nhiều module.
Artifact	Sản phẩm cụ thể được tạo và triển khai: bundle JavaScript, executable, container image hoặc gói cấu hình.
Node	Tài nguyên tính toán hoặc môi trường thực thi có thể chứa artifact.
Class / object	Class là kiểu định nghĩa cấu trúc và hành vi; object là một thể hiện cụ thể của kiểu đó.
Interface	Hợp đồng mà bên sử dụng dựa vào; không phải giao diện màn hình.

5 Layer, tier và module nghiệp vụ

Layer là phân chia trách nhiệm trong thiết kế/mã nguồn. **Tier** là phân chia nơi thực thi. Một backend có presentation, application, domain và infrastructure vẫn có thể chạy trong một process trên một máy.



Hình 1: *Nhiều layer không có nghĩa nhiều máy chủ*

Module nghiệp vụ là cách chia theo chức năng, ví dụ đăng ký, thanh toán, thông báo. Layer chia theo vai trò kỹ thuật. Hai cách có thể kết hợp: bên trong mỗi module có application và domain, còn adapter dùng chung được tổ chức riêng khi hợp lý.

6 Một số lựa chọn kiến trúc

Layered architecture

Tách phần nhận yêu cầu, xử lý nghiệp vụ và truy cập dữ liệu. Quy định rõ tầng nào được gọi tầng nào. Không để controller chứa toàn bộ quy tắc rồi coi service là lớp chuyển tiếp rỗng.

Modular monolith

Một ứng dụng triển khai chung, bên trong có các module nghiệp vụ có ranh giới. Hữu ích để bảy người chia việc mà chưa phải vận hành bảy dịch vụ qua mạng. Đổi lại, cần giữ kỷ luật phụ thuộc; đặt tên thư mục riêng chưa tự tạo ra ranh giới tốt.

Microservices

Các dịch vụ được triển khai độc lập và trao đổi qua hợp đồng mạng. Đổi lại là xử lý lỗi mạng, dữ liệu phân tán, giám sát và triển khai phức tạp hơn. “Bảy phân hệ” không suy ra “bảy microservices”. Chỉ chọn khi có nhu cầu và năng lực vận hành tương ứng.

MVC và Clean Architecture không phải hai tên thay thế cho mọi thứ

MVC phân biệt model, view và controller ở một phạm vi tổ chức giao diện/ứng dụng. Clean Architecture nhấn mạnh ranh giới và chiều phụ thuộc hướng về chính sách nghiệp vụ. Không cần áp dụng đầy đủ mọi mẫu; cần giải thích cấu trúc thực sự dùng.

7 Đánh giá một quyết định thiết kế

Cohesion cao: các trách nhiệm trong một khối liên quan chặt với nhau. **Coupling thấp**: khối không phải biết quá nhiều chi tiết bên trong của khối khác. Không phải

càng nhiều class càng tốt; chia thành hàng chục lớp chỉ chuyển tiếp lời gọi cũng làm khó hiểu.

Một quyết định nên có bốn ý: vấn đề cần giải quyết, lựa chọn, lý do và đánh đổi. Ví dụ: "Dùng optimistic version để ngăn hai thao tác ghi đè. Đổi lại, request có thể bị conflict và phải đọc lại dữ liệu." Câu đó có giá trị hơn "dùng kiến trúc tối ưu, linh hoạt".

Architectural driver

Đây là yêu cầu ảnh hưởng mạnh đến kiến trúc: cạnh tranh tài nguyên, khả năng phục hồi, độ trễ, bảo mật hoặc cách ly mô phỏng. Không phải mọi màu nút hay tên màn hình đều là architectural driver.

Giữ phân biệt thiết kế và bằng chứng

"Sẽ dùng transaction" là phương án. "Đã kiểm thử 50 yêu cầu đồng thời và chỉ một yêu cầu thành công" là kết quả cần bằng chứng chạy. Một sơ đồ không chứng minh phần mềm đã được triển khai hoặc đạt hiệu suất.

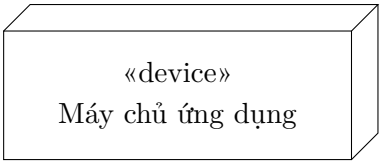
II Deployment View

1 Lý thuyết từ đầu

Deployment View mô tả **cấu hình thực thi**: phần mềm nào chạy trong môi trường nào, trên tài nguyên nào và các nơi đó kết nối ra sao. Người đọc phải phân biệt được trình duyệt của người dùng, backend, nơi lưu dữ liệu và hệ thống ngoài.

Nó không phải sơ đồ thứ tự xử lý một use case. Trục trên–dưới của hình không biểu diễn thời gian; đường HTTP không cho biết phải kiểm tra quyền trước hay sau. Thứ tự đó thuộc sequence/activity.

2 Node và device

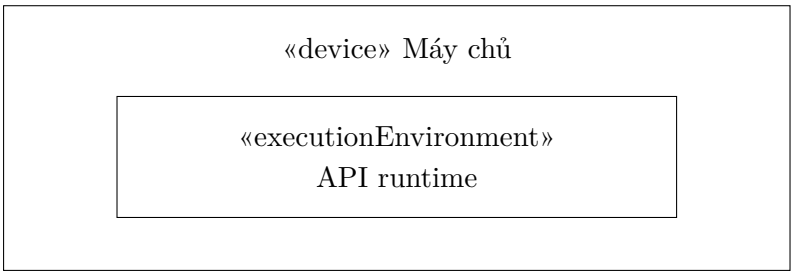


Hình 2: *Hộp có chiều sâu là cách biểu diễn node*

Node là một tài nguyên tính toán của mô hình triển khai. `«device»` thường dùng cho thiết bị có năng lực xử lý, như máy người dùng hoặc máy chủ. Với máy ảo cần ghi rõ đó là tài nguyên ảo, không gọi mọi hình hộp là phần cứng vật lý.

Nếu nói về một instance cụ thể có thể dùng dạng `app01:ApplicationServer`; nếu đang thiết kế một loại node thì chỉ cần tên loại. Tên node cần thể hiện nó là nơi chạy, không phải tên chức năng như "Đặt xe".

3 Execution environment

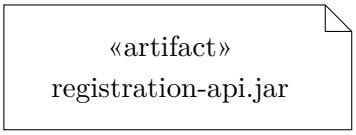


Hình 3: *Môi trường thực thi có thể nằm trong một node khác*

Execution environment là môi trường phần mềm cung cấp khả năng chạy artifact: JVM, một runtime ứng dụng hoặc môi trường container được mô hình hóa phù hợp. Nó không phải một thuật ngữ khác của class.

Container có thể được biểu diễn như một môi trường thực thi có chú thích rõ. Container image là artifact được tạo trước khi chạy; container đang chạy là một thể hiện runtime. Không ghi cùng một ô vừa là image vừa là process mà không giải thích.

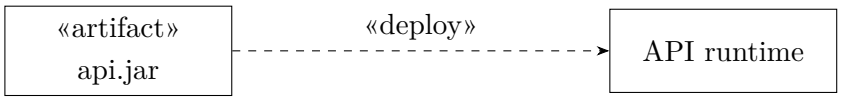
4 Artifact và quan hệ triển khai



Hình 4: *Artifact là sản phẩm phần mềm cụ thể*

Artifact có thể là executable, thư viện, gói web hoặc tập lệnh cấu hình. Component "Đăng ký" là trách nhiệm logic; file `registration-api.jar` là một sản phẩm có thể chứa nhiều component.

Có hai cách thường dùng: đặt artifact bên trong node đích hoặc vẽ dependency nét đứt có nhãn `«deploy»` từ artifact đến deployment target. Phải thống nhất cách đọc trong chú giải.

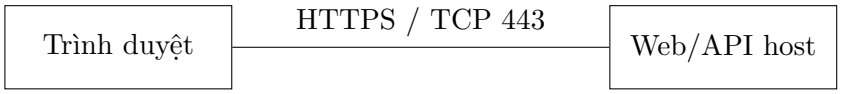


Hình 5: *Artifact được triển khai tới môi trường đích*

Manifestation

Artifact có thể hiện thực một component logic. Quan hệ `«manifest»` nối artifact tới phần tử mà nó biểu hiện. Nó khác deploy: một bên nói "sản phẩm này hiện thực phần nào", bên kia nói "sản phẩm này được đặt ở đâu".

5 Communication path và protocol



Hình 6: *Đường nối liền giữa các node biểu diễn đường giao tiếp*

Communication path thường là đường liền. Ghi protocol và port khi chúng giúp tái tạo cấu hình. Nếu thêm đầu mũi tên để chỉ bên khởi tạo kết nối thì phải nói rõ; HTTP response vẫn có thể đi ngược lại trên cùng kết nối.

Port TCP như 443 khác UML port hình ô vuông ở biên component. Một bên là địa chỉ truyền thông của runtime; bên kia là điểm tương tác logic của component.

6 System boundary và hệ thống ngoài

Vẽ rõ phần nào nhóm triển khai và phần nào sử dụng từ ngoài. Dịch vụ bản đồ không biến thành class nội bộ chỉ vì backend gọi nó. Database nội bộ cũng không trở thành actor nghiệp vụ chỉ vì nằm trên một máy khác.

Boundary triển khai và trust boundary không nhất thiết trùng nhau. Kết nối từ trình duyệt tới API cần xác thực và kiểm tra quyền; database thường không mở trực tiếp cho trình duyệt. Một đường nối trên sơ đồ không phải là quyết định cho phép truy cập vô điều kiện.

7 Local, Docker và production

Môi trường	**Thông tin cần làm rõ**
Local	Chạy chương trình nào, trên máy nào, cổng nào và dữ liệu được khởi tạo ở đâu.
Docker	Container, image, network, volume, cấu hình và cách kiểm tra dịch vụ đã sẵn sàng.
Production	Điểm vào HTTPS, nơi giữ secret, dữ liệu bền vững, backup, log và hành vi khi dịch vụ lỗi.

Docker không đồng nghĩa cloud, và thêm logo cloud không làm bản thiết kế trở thành production. Chỉ mô tả nhà cung cấp hoặc dịch vụ cụ thể là phần nhóm sẽ dùng khi đã có quyết định tương ứng.

Các container trong cùng một máy không tự tạo ra khả năng chịu lỗi nếu máy đó sập. Chạy nhiều replica cũng không tự giải quyết cạnh tranh đặt tài nguyên nếu kiểm tra được giữ trong bộ nhớ riêng của từng replica.

8 Cách tự dựng một Deployment View

1. Xác định người sử dụng và hệ thống ngoài cần kết nối.
2. Liệt kê artifact cần chạy: web bundle, API, worker nếu thực sự có, cấu hình dữ liệu.
3. Đặt từng artifact vào runtime và host phù hợp.
4. Nối các nơi có giao tiếp thật; ghi protocol, port và boundary cần bảo vệ.
5. Mô tả cách khởi động, cấu hình, dữ liệu và kiểm tra sẵn sàng.
6. Thử các tình huống mất mạng, lỗi nguồn ngoài và khởi động lại; giải thích hệ thống giữ gì và mất gì.

Không cần liệt kê mọi chi tiết phần cứng chưa quyết định. Mức chi tiết đúng là mức giúp người khác hiểu và tái tạo cách chạy, không phải mức có nhiều biểu tượng nhất.

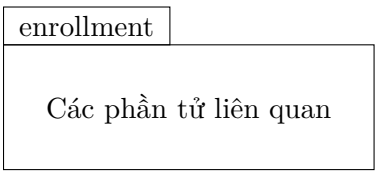
III Development/Implementation View

1 Lý thuyết từ đầu

View này trả lời: mã nguồn được chia thành những phần nào, trách nhiệm của từng phần là gì và phần nào được phép phụ thuộc phần nào. Nó giúp một thành viên biết nơi cần viết code và hợp đồng phải giữ khi tích hợp với người khác.

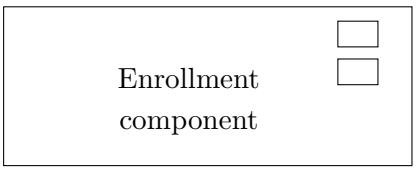
Không nhầm "implementation" với việc phải hoàn tất code ngay trong tài liệu. Đây là thiết kế cấu trúc hiện thực. Một sơ đồ thư mục có thể hỗ trợ giải thích, nhưng cần thêm trách nhiệm và các quan hệ mà tên thư mục không thể tự nói lên.

2 Package và component



Hình 7: *Package là cách tổ chức các phần tử theo một phạm vi*

Package thường biểu diễn một module hoặc namespace. Đường bao package không tự bảo đảm tính đóng gói: vẫn cần quy định phần tử nào công khai, phần tử nào nội bộ và ai được import.



Hình 8: *Component có thể dùng biểu tượng UML nhỏ ở góc hoặc stereotype component*

Component cần có trách nhiệm và interface cung cấp/sử dụng. Một component có thể được hiện thực bởi nhiều class. Một artifact cũng có thể đóng gói nhiều component; không bắt buộc một–một.

3 Provided interface và required interface

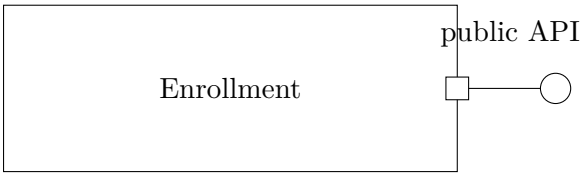


Hình 9: *Lollipop biểu diễn cung cấp interface; socket biểu diễn cần interface*

Provided interface là năng lực component cung cấp; required interface là năng lực nó cần từ bên khác. Khi nối cặp tương thích có thể dùng assembly connector. Không coi mọi REST endpoint là một vòng tròn bắt buộc phải vẽ riêng.

Interface ở mức component có thể là nhóm chức năng. Interface ở class diagram liệt kê operation cụ thể. Hai mức cần tương thích nhưng không phải cùng một hình vẽ.

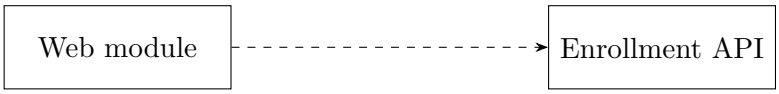
4 Port ở biên component



Hình 10: *Ô vuông nhỏ trên biên là một port logic*

Port gom một điểm tương tác của component với môi trường. Không cần vẽ port cho mọi mũi tên. Không gọi ô vuông này là cổng mạng 8080 trừ khi đang ghi một quy ước bổ sung rất rõ.

5 Dependency và chiều mũi tên



Hình 11: *Nét đứt đi từ bên sử dụng đến phần mà nó phụ thuộc*

Nếu Web module cần hợp đồng của Enrollment API để được viết hoặc kiểm tra kiểu, mũi tên dependency hướng từ Web tới API. Đây không phải chiều dữ liệu duy nhất khi chương trình chạy: API còn trả response về Web.

Vòng phụ thuộc A tới B rồi B tới A làm thay đổi và kiểm thử khó hơn. Không phải mọi vòng đều không thể hiện thực, nhưng cần lý do rõ ràng thay vì coi đó là tình trạng mặc định.

6 Bốn vai trò thường gặp trong backend

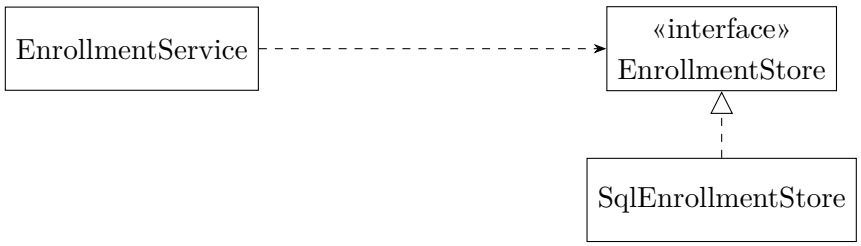
Vai trò	**Trách nhiệm và giới hạn**
Presentation	Nhận request, kiểm tra hình thức dữ liệu, chuyển lỗi sang response; không là nơi duy nhất giữ quy tắc nghiệp vụ.

Vai trò	Trách nhiệm và giới hạn
Application	Điều phối một use case, kiểm tra quyền, mở transaction, gọi đối tượng miền và adapter; không đồng nghĩa mỗi thao tác đều cần một service riêng.
Domain	Đối tượng, invariant và chính sách nghiệp vụ; tránh phụ thuộc chi tiết HTTP, SQL hoặc giao diện.
Infrastructure	Cách truy cập dữ liệu, gửi/nhận sự kiện, tích hợp nguồn ngoài; hiện thực các hợp đồng mà ứng dụng cần.

Ví dụ một yêu cầu đăng ký được controller nhận, application service điều phối, Course kiểm tra chỗ và adapter lưu thay đổi. Gọi thẳng SQL từ một nút giao diện làm ranh giới khó kiểm soát và bỏ qua quyền ở server.

7 Dependency inversion: vì sao cần interface?

Giả sử EnrollmentService gọi trực tiếp một class SQL cụ thể. Khi thay dữ liệu bằng bộ nhớ để học hoặc kiểm thử, service phải sửa theo chi tiết lưu trữ. Có thể tách một interface EnrollmentStore do phía ứng dụng định nghĩa nhu cầu, rồi để SqlEnrollmentStore hiện thực.



Hình 12: *Cả service và adapter cụ thể dựa vào hợp đồng thay vì service dựa vào SQL*

Khi chạy, object service vẫn gọi object adapter đã được truyền vào. **Chiều phụ thuộc mã nguồn và đường gọi runtime là hai chuyện khác nhau**. Dependency injection là cách cung cấp object phù hợp; có thể truyền qua constructor, không bắt buộc dùng framework.

Không tạo interface cho mọi class chỉ để hình trông đầy đủ. Tách ở những ranh giới cần thay cách hiện thực, kiểm thử độc lập hoặc che giấu chi tiết quan trọng.

8 Repository, service, DTO và adapter

Repository / data-store interface

Định nghĩa cách truy vấn và lưu các đối tượng cần thiết. Chỉ nói rõ method là chưa đủ: phải biết nó đọc snapshot hay trạng thái hiện tại, có khóa không và nằm

trong transaction nào.

DTO

Data Transfer Object mang dữ liệu qua một ranh giới, ví dụ EnrollmentRequest hoặc EnrollmentResult. DTO không mặc nhiên là entity lưu trong database. Hạn chế trả trực tiếp object có password hash, quyền nội bộ hoặc thông tin nhạy cảm ra UI.

Application service và domain service

Application service điều phối use case. Domain service chứa chính sách nghiệp vụ không hợp lý khi gắn vào một entity đơn lẻ. Không cần tách domain service nếu hành vi đặt ngay trên entity đã rõ và dễ kiểm thử.

Adapter

Chuyển đổi hợp đồng nội bộ sang cách giao tiếp cụ thể của SQL, HTTP hoặc luồng sự kiện. Nếu nguồn ngoài đổi định dạng pin từ 0–1 sang 0–100, phần chuyển đổi không nên bị lặp ở mọi UI.

9 Từ view đến cấu trúc thư mục

Một cách tổ chức minh họa, không bắt buộc theo ngôn ngữ cụ thể:

```
src/
  enrollment/
    presentation/    # controller, request/response
    application/     # use-case service, store contract
    domain/          # Course, Registration, policy
    infrastructure/  # SQL store implementation
  shared/            # types/contracts truly shared
  bootstrap/         # construct objects, configuration
tests/
```

Tên thư mục chỉ có ích khi tương ứng trách nhiệm thực tế. `shared` không phải nơi đổ mọi class chưa biết đặt ở đâu. Một lớp dùng chung cần người quản lý định nghĩa, tránh mỗi phân hệ viết lại cùng kiểu dữ liệu theo một cách khác.

10 Cross-cutting concerns

Xác thực, phân quyền, audit, transaction, cấu hình, thời gian và lỗi được dùng ở nhiều module. Có thể có cơ chế chung, nhưng từng use case vẫn phải xác định đối tượng/phạm vi cần kiểm tra và dữ liệu nào được phép thay đổi.

Log kỹ thuật phục vụ chẩn đoán; audit ghi nhận ai thực hiện thao tác nghiệp vụ nào và tác động ra sao. Không ghi password hoặc token vào hai loại log này. Một thông báo "lỗi" không thay thế hợp đồng lỗi rõ ràng giữa module.

IV Class diagram: lý thuyết và từng ký hiệu

1 Class diagram dùng để làm gì?

Class diagram mô tả **cấu trúc tĩnh**: các kiểu đối tượng, thuộc tính, operation và quan hệ. "Tĩnh" không có nghĩa dữ liệu không đổi; nghĩa là hình không diễn tả thứ tự các lần gọi theo thời gian.

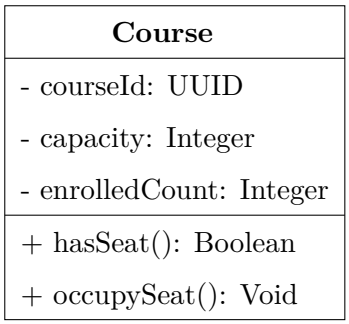
Use case là mục tiêu tương tác, không phải class. Screen là giao diện, không phải entity. Database table là cấu trúc lưu trữ, không tự động là toàn bộ class diagram. Thiết kế cần cả các lớp điều phối và hợp đồng nếu chúng thực sự có trách nhiệm trong hệ thống.

2 Class và object

Class Course định nghĩa những course có thuộc tính và hành vi nào. Hai object cụ thể có thể là `courseA:Course` và `courseB:Course`; chúng có cùng kiểu nhưng capacity hoặc enrolledCount khác nhau.

Trong object diagram, tên object thường được gạch dưới. Trong class diagram, tên Course không gạch dưới; gạch dưới một member lại mang nghĩa static. Cần đọc đúng vị trí ký hiệu.

3 Hộp class và ba ngăn



Hình 13: *Ba ngăn lần lượt là tên, thuộc tính và operation*

Ngăn tên trả lời "đây là kiểu gì". Ngăn thuộc tính ghi trạng thái và dữ liệu quan trọng. Ngăn operation ghi các hành vi mà đối tượng cung cấp. Có thể ẩn ngăn trống hoặc bớt chi tiết trên hình tổng quan nếu nói rõ phạm vi.

Operation là hợp đồng thao tác ở mức mô hình; method là cách hiện thực hành vi đó trong code. Trong bài, "method descriptions" là phần giải thích đầy đủ cho các thao tác được vẽ, không chỉ chép lại tên.

4 Attribute: tên, kiểu, số lượng và giá trị mặc định

Cú pháp thường dùng:

```
visibility name: Type [multiplicity] = defaultValue
- expiresAt: DateTime
- roles: Role [0..*]
- retryCount: Integer = 0
/remainingSeats: Integer
```

Không cần dùng tất cả thành phần trên mọi dòng. Kiểu dữ liệu phải có ý nghĩa: thời gian dùng DateTime/Instant có quy ước múi giờ; số tiền phải có đơn vị và cách làm tròn; phần trăm pin cần quy định thang đo.

Dấu / trước remainingSeats nghĩa là thuộc tính **dẫn xuất**, có thể tính từ dữ liệu khác. Nếu vẫn lưu nó, phải thiết kế cách đồng bộ; không được giả định nó luôn đúng mà không có cơ chế.

5 Visibility: ai được truy cập?

Dấu	Tên	Ý nghĩa
+	public	Có thể dùng từ bên ngoài theo hợp đồng công khai.
-	private	Chỉ bên trong classifier/lớp sở hữu theo quy tắc mô hình.
#	protected	Dành cho lớp và các lớp chuyên biệt hóa theo quy tắc ngôn ngữ.
~	package	Phạm vi package; cách ánh xạ sang code tùy ngôn ngữ.

Đây không phải quyền của Sinh viên, Admin hay Operator. Một method public vẫn phải kiểm tra quyền nghiệp vụ khi thực hiện yêu cầu được bảo vệ. JavaScript, Java và C# cũng không ánh xạ mọi mức visibility giống hệt nhau.

Encapsulation không chỉ là thêm private: đối tượng cần kiểm soát cách thay đổi trạng thái. Nếu ai cũng được đặt status tùy ý qua setter thì invariant vẫn có thể bị phá.

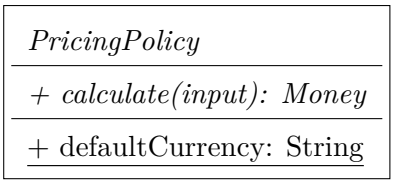
6 Operation và signature

```
+ enroll(request: EnrollmentRequest): EnrollmentResult
+ findById(id: UUID): Registration [0..1]
- validateCapacity(course: Course): Void
```

Signature nêu tên, tham số và kiểu trả về. `[0..1]` cho biết có thể không tìm thấy một đối tượng. Có thể dùng ký hiệu `Registration?` như quy ước ngôn ngữ, nhưng cần giải thích rằng đây là dạng optional/nullable, không tự trộn hai cách viết.

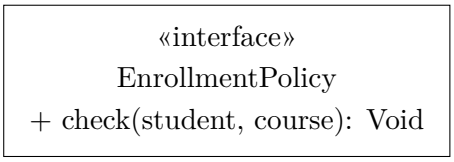
Đặt tên bằng động từ và mục tiêu, như confirm, cancel hoặc calculateDuration. Không dùng một method `handle()` cho mọi thứ rồi để người đọc tự đoán nó nhận hoặc thay đổi gì.

7 Static, abstract và interface



Hình 14: *Chữ nghiêng thường biểu diễn abstract; gạch dưới biểu diễn static*

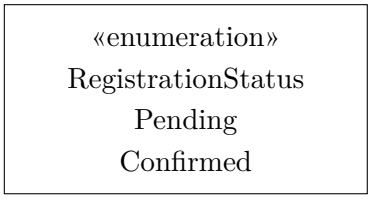
Static gắn với classifier, không với từng object. Abstract chỉ phần tử chưa có hiện thực đầy đủ hoặc không được tạo instance trực tiếp theo mô hình. Trong code PlantUML có thể dùng `{static}` và `{abstract}` để diễn đạt.



Hình 15: *Interface định nghĩa hợp đồng, không phải một màn hình*

Một class hiện thực interface phải đáp ứng hợp đồng của nó. Không tạo instance trực tiếp từ một interface chỉ có khai báo operation; ở runtime cần một implementation cụ thể.

8 Stereotype, enumeration và constraint



Hình 16: *Enumeration gom các giá trị trạng thái được phép*

Stereotype viết trong cặp dấu « », như `«interface»`. Các nhãn entity, service, DTO hoặc repository trong tài liệu có thể là quy ước nhóm, không có nghĩa mọi nhãn đó đều là một kiểu ký hiệu chuẩn riêng của UML.

Constraint thường ghi trong ngoặc nhọn hoặc note, ví dụ `{enrolledCount <= capacity}`. Invariant là điều kiện phải giữ ở các trạng thái hợp lệ, không phải công việc xảy ra trước công việc khác.

9 Association: các đối tượng có liên hệ



Hình 17: *Một Student có thể liên hệ nhiều Registration; mỗi Registration thuộc một Student*

Association là quan hệ giữa các instance. Đường liền không tự có nghĩa gửi request, gọi method hoặc có foreign key. Vai trò ở đầu quan hệ có thể ghi để làm rõ, chẳng hạn registrations và student.

Không bắt buộc thêm field registrations nếu hình đã biểu diễn quan hệ và chưa muốn khóa cách lưu. Khi hiện thực, quan hệ có thể được giữ bằng tham chiếu object, ID hoặc truy vấn theo khóa; phải giữ ngữ nghĩa của mô hình.

10 Multiplicity: đọc số ở đầu nào?

Số **đặt gần B** cho biết với **một A** đang xét có bao nhiêu B liên hệ qua quan hệ đó. Sau đó đổi chiều để đọc đầu còn lại. Không đọc cả hai số theo một chiều trái–phải duy nhất.

Ký hiệu	Ý nghĩa
`1`	Đúng một.
`0..1`	Có thể không có, hoặc có một.
`0..*`, `*`	Không hoặc nhiều, không đặt giới hạn trên trong mô hình.
`1..*`	Ít nhất một.
`2..5`	Từ hai đến năm.

Multiplicity phải xét **phạm vi thời gian và loại quan hệ**. Một xe có nhiều lượt đặt trong lịch sử nhưng tối đa một lượt đặt hiệu lực. Vì vậy có thể dùng association 0..* cho lịch sử và thêm constraint về active reservation; không sửa multiplicity lịch sử thành 0..1 để che mất dữ liệu cũ.

11 Navigability: biết cách đi tới đối tượng khác



Hình 18: *Đầu mũi tên mở biểu diễn khả năng điều hướng tới Student*

Navigability thể hiện khả năng từ một đầu đi tới đầu kia theo mô hình. Nó không phải đường đi của HTTP request. Quan hệ không có đầu mũi tên không nên bị tự diễn giải là code bắt buộc có hai field tham chiếu qua lại; cần nêu quy ước điều hướng nếu điều đó quan trọng.

12 Aggregation: quan hệ toàn thể–bộ phận yếu



Hình 19: *Hình thoi rỗng ở phía toàn thể*

Aggregation thường dùng khi muốn nhấn mạnh tập hợp/toàn thể và phần tử vẫn có thể tồn tại độc lập. Shared aggregation trong UML không tự quy định một chính sách vòng đời thống nhất cho mọi mô hình; cần giải thích ý nghĩa cụ thể. Nếu chỉ biết A liên hệ B mà chưa có ngữ nghĩa đặc biệt, association thường rõ hơn.

13 Composition: sở hữu bộ phận chặt hơn



Hình 20: *Hình thoi đặc đặt ở đầu composite, không phải ở đầu bộ phận*

Trong composite aggregation, một part không thuộc đồng thời nhiều composite owner. Khi composite bị xóa, các part còn thuộc nó cũng bị xóa theo ngữ nghĩa mô hình; part có thể được tách trước đó nếu mô hình cho phép. Đây là ràng buộc về đối tượng và vòng đời, không chỉ là "A có danh sách B".

Trong phần mềm có lịch sử hoặc yêu cầu audit, thao tác xóa có thể bị cấm hoặc dùng lưu trữ/ẩn thay vì xóa vật lý. Không suy ra rằng cứ vẽ composition là phải đặt SQL `ON DELETE CASCADE`. Với Hub và xe có thể điều chuyển, không nên chọn composition chỉ vì xe đang nằm ở Hub.

14 Generalization: kế thừa kiểu

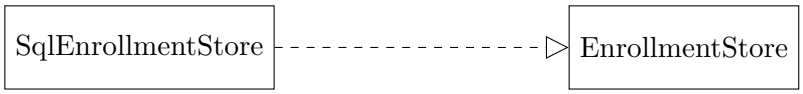


Hình 21: *Đường liền, tam giác rỗng hướng tới kiểu tổng quát*

Generalization nói rằng kiểu chuyên biệt có thể được xem là một trường hợp của kiểu tổng quát. Mũi tên hướng từ con lên cha. Không dùng kế thừa để nói "được cấp nhiều quyền hơn" hoặc "được thao tác lên".

Đặc biệt, Admin và Operator là các vai trò quyền hạn, không mặc nhiên là hai class có quan hệ cha–con. Một account có thể gắn các assignment theo thiết kế quyền mà không cần một class cho mỗi role.

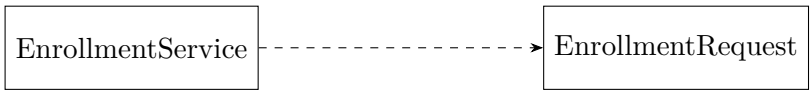
15 Realization: hiện thực hợp đồng



Hình 22: *Nét đứt, tam giác rỗng hướng tới interface*

SqlEnrollmentStore hiện thực EnrollmentStore. Khác generalization ở kiểu đường nét đứt, và khác dependency ở đầu tam giác rỗng. Không nối ngược chiều chỉ vì interface được viết trước trong tài liệu.

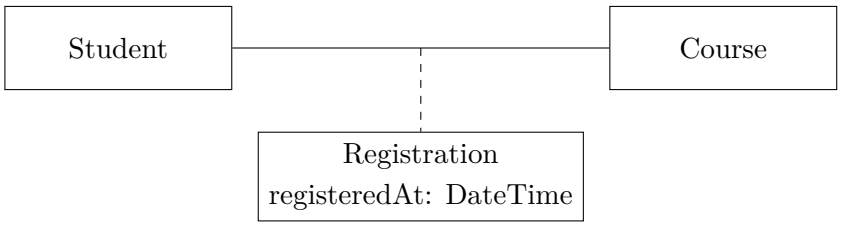
16 Dependency: cần dùng một kiểu khác



Hình 23: *Dependency là nét đứt với đầu mũi tên mở hướng tới bên được dùng*

Nếu một class chỉ nhận một kiểu làm tham số hoặc tạo nó trong một thao tác, dependency có thể đủ. Nếu giữ một tham chiếu lâu dài như store của service, association có thể thể hiện cấu trúc rõ hơn. Không cần vẽ hai cạnh trùng nhau chỉ để biểu diễn cùng một ý.

17 Association class: dữ liệu thuộc về một quan hệ



Hình 24: *Association class gắn bằng nét đứt vào chính đường association*

Đăng ký giữa Student và Course có thể có thời điểm và trạng thái riêng. Có thể mô hình bằng association class, hoặc dùng class Registration liên kết hai đầu như ví dụ trong tài liệu. Không dùng đồng thời cả hai cách cho cùng quan hệ rồi coi là hai thực thể khác nhau.

18 Chọn class từ đâu và tránh vẽ quá nhiều

Lấy danh từ nghiệp vụ để tìm entity/value object, lấy trách nhiệm trong use case để tìm service, lấy các ranh giới để tìm controller, interface và adapter. Không phải mọi danh từ đều thành class: "nút áp dụng" có thể chỉ là chi tiết UI.

Với hình tổng thể, giữ các lớp/quan hệ chính và tách hình chi tiết theo module. Không ép một class diagram cho mỗi UC nếu chúng dùng chung một tập lớp. Mỗi class và operation cần xuất hiện ở nơi có trách nhiệm rõ, không nhân bản chỉ vì nhiều người dùng tới.

V Thiết kế lớp và mô tả phương thức

1 Vì sao tên method chưa đủ?

Hai người có thể cùng viết `cancelReservation(id)` nhưng một người hủy mọi trạng thái, người kia chỉ hủy khi Confirmed. Nếu không có hợp đồng, tích hợp vẫn sai dù tên giống hệt. Method description xác định những điều bên gọi được dựa vào và bên hiện thực phải giữ.

Không phải viết lại toàn bộ main flow vào mọi method. Một use case thường đi qua nhiều method; mỗi method mô tả đúng phần trách nhiệm của mình. Ngược lại, một method có thể được nhiều UC dùng chung.

2 Mô tả class trước khi mô tả operation

Một class nên có: tên và vai trò, trách nhiệm chính, thuộc tính quan trọng, invariant và các quan hệ cần giữ. Nếu khó nói trách nhiệm bằng một câu rõ nghĩa, class có thể đang ôm quá nhiều việc hoặc tồn tại mà không có lý do.

Ví dụ Course giữ khả năng tiếp nhận đăng ký và kiểm soát số người đã nhận. Invariant: capacity không âm, enrolledCount không âm và không vượt capacity. Course không cần biết token HTTP được đặt trong header nào.

3 Những trường cần có trong một hợp đồng

Trường	**Nội dung cần quyết định**
Owner và signature	Class sở hữu; tên method; tên, kiểu tham số và kiểu trả về.
Mục đích	Một trách nhiệm kiểm chứng được, không chỉ viết “xử lý dữ liệu”.
Input	Đơn vị, miền giá trị, trường bắt buộc, nguồn danh tính và trường có thể thiếu.
Precondition	Điều kiện cần đúng trước khi thao tác được thực hiện hợp lệ.
Postcondition	Điều được bảo đảm sau thành công; nêu cả trạng thái và giá trị trả về.
Error/failure	Trường hợp không thực hiện được và trạng thái phải được giữ sau lỗi.
Side effect	Dữ liệu ghi, log/audit, sự kiện hoặc lệnh ra ngoài; ghi “không” nếu chỉ đọc.

Trường	Nội dung cần quyết định
Concurrency	Transaction, khóa/version, điều kiện cập nhật và cách tránh ghi đè/đặt trùng.
Retry và idempotency	Có thể thử lại không; nhận diện yêu cầu cũ thế nào; khi nào không được tự lặp.
Liên hệ và kiểm chứng	UC/BR liên quan, caller chính và test kiểm chứng hành vi.

Không bắt buộc mọi method đơn giản có một bảng dài bằng nhau. Một getter thuần có thể mô tả gọn. Nhưng không được bỏ qua transaction hoặc side effect của thao tác đặt, xác nhận và kết thúc chỉ để bảng ngắn.

4 Precondition, validation, guard và invariant

Precondition không miễn kiểm tra ở server

"Lượt đặt thuộc người dùng" là điều kiện thực hiện; không có nghĩa server tin vào ID do UI gửi. Điểm vào hệ thống vẫn xác minh điều kiện. Bên trong một helper private có thể dựa vào caller, nhưng phải nói rõ phạm vi tin cậy đó.

Validation

Là hành động kiểm tra. Cần tách hình thức dữ liệu, quyền truy cập và điều kiện nghiệp vụ. Một số hợp lệ về cú pháp vẫn có thể bị từ chối vì không đủ chỗ hoặc không có quyền.

Guard

Là điều kiện cho một chuyển trạng thái hoặc nhánh. Ví dụ chỉ Confirmed mới được đổi sang Expired khi đã tới expiresAt và chưa bàn giao. Method làm transition cần giữ chính guard đó, không chỉ gán một chuỗi status.

Invariant

Phải giữ ở các trạng thái hợp lệ của đối tượng/miền. Nếu kiểm tra rồi cập nhật bằng hai thao tác độc lập, một luồng khác có thể chen vào làm invariant bị phá. Hợp đồng phải nói cơ chế bảo vệ chứ không chỉ nêu mong muốn.

5 Return value và lỗi

Có thể dùng exception có kiểu, result object hoặc mã lỗi nhất quán; không cần ép mọi ngôn ngữ theo một mẫu. Phải phân biệt không tìm thấy, không đủ điều kiện, không có quyền, xung đột phiên bản và lỗi hạ tầng khi bên gọi cần xử lý khác nhau.

Ví dụ một `EnrollmentResult` có mã Confirmed, Full, Duplicate, Inactive hoặc Conflict. Một lỗi kết nối dữ liệu không nên bị trả thành Full vì bên gọi sẽ hiểu sai tình trạng khóa học.

Không trả `null` cho tất cả lỗi rồi bắt UI tự suy ra. Không đưa stack trace hoặc chi tiết database nhạy cảm ra response công khai.

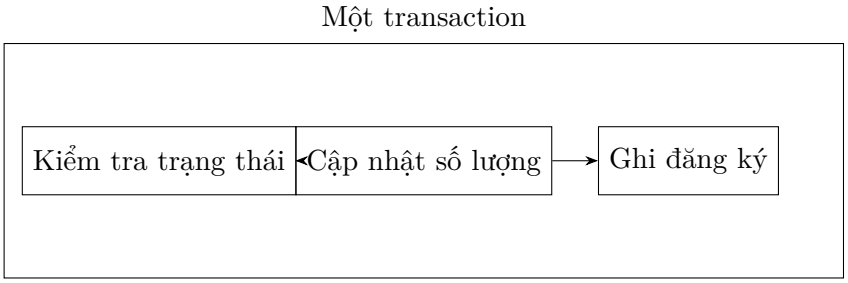
6 Query và command

Query lấy thông tin; **command** thay đổi trạng thái nghiệp vụ. Một query có thể ghi log kỹ thuật hoặc metric, nhưng không nên âm thầm tạo booking hay phân bổ lại lịch sạc. Nếu có side effect, phải nêu rõ và đánh giá việc gọi lặp.

Ví dụ xem lịch sạc không được tự điều chỉnh một phiên đang Charging chỉ vì người dùng làm mới màn hình. Tên method bắt đầu bằng get không tự chứng minh nó chỉ đọc; cần kiểm tra nội dung hợp đồng.

7 Transaction và thay đổi nguyên tử

Transaction gom các thay đổi cần thành công hoặc thất bại cùng nhau. Ví dụ vừa tăng số người đăng ký vừa tạo Registration: không được tăng số lượng rồi mất bản ghi đăng ký nếu bước sau thất bại.



Hình 25: *Các bước cần cùng giữ một kết quả nhất quán*

Atomicity không có nghĩa mọi vấn đề cạnh tranh đã được giải quyết. Còn phải chọn isolation, khóa, điều kiện ghi hoặc ràng buộc phù hợp. Chỉ đặt hai lệnh ghi trong transaction mà không bảo vệ bước kiểm tra vẫn có thể sai.

Lệnh gửi ra thiết bị ngoài không tự rollback theo transaction SQL. Nếu gửi lệnh rồi mất phản hồi, cần đọc lại trạng thái hoặc có quy trình hòa giải. Đừng viết “rollback” như một thao tác có thể đưa thế giới vật lý về trước.

8 Ba cơ chế cạnh tranh cần hiểu

1. **Khóa**: ngăn thao tác cạnh tranh sửa cùng tài nguyên trong một phạm vi. Phải biết khóa ở đâu, giữ tới khi nào và có bao phủ nhiều process không.

2. **Optimistic version / compare-and-set**: chỉ ghi nếu version hoặc trạng thái vẫn như lúc đọc. Không đạt điều kiện thì trả conflict và không ghi một phần.
3. **Unique constraint**: nơi lưu dữ liệu từ chối những tổ hợp không được trùng. Với ràng buộc "đang hiệu lực", cần mô hình phù hợp; unique toàn bộ lịch sử thường quá chặt.

Có thể kết hợp các cơ chế. Mutex trong một process không bảo vệ hai backend process khác nhau. Một file JSON cũng không tự có transaction an toàn chỉ vì thao tác ghi tương đối nhanh.

9 Idempotency: thử lại mà không tạo tác động trùng

Idempotent nói về hiệu ứng: gửi lại cùng một thao tác hợp lệ theo hợp đồng không nhân đôi tác động dự định. Response không nhất thiết giống từng byte. Một yêu cầu mới khác người dùng hoặc khác payload không được coi là yêu cầu cũ chỉ vì tình cờ dùng cùng ID.

Với yêu cầu có thể mất phản hồi, lưu request ID cùng kết quả trong cùng transaction khi cần. Khi thử lại, đối chiếu danh tính và nội dung; trả kết quả đã xác nhận hoặc báo key dùng sai. Không tạo booking lần hai chỉ vì UI chưa nhận được response lần đầu.

10 Thời gian, tác vụ nền và dữ liệu ngoài

Nêu rõ expiresAt tính từ thời điểm nào, đơn vị gì và dùng đồng hồ nào. Trong code kiểm thử có thể truyền Clock thay vì chờ thật 10 phút. Khi sự kiện hết hạn và nhận xe cạnh tranh, method cần kiểm tra trạng thái hiện tại một cách nhất quán.

Dữ liệu bên ngoài cần quy định event ID, tài nguyên, thời điểm đo/nhận, thứ tự/version và độ hợp lệ. Một sự kiện đến muộn không được vô điều kiện ghi đè trạng thái mới. Nguồn ngoài không được tự mở tài nguyên đã Inactive bằng một giá trị Available.

11 Từ sequence tới method descriptions

Đọc từng thông điệp nghiệp vụ trong sequence và xác định receiver chịu trách nhiệm. Sau đó chốt signature, chuyển điều kiện nhánh thành validation/guard, xác định dữ liệu được ghi và nhóm các ghi cần nguyên tử. Cuối cùng đối chiếu transition trong state chart.

Không chuyển mọi mũi tên UI thành một method domain. Cũng không bỏ tác vụ tự động chỉ vì nó không có nút bấm. Với method xuất hiện ở nhiều UC, mô tả hợp đồng một lần tại class chuẩn rồi dẫn chiếu các nơi dùng.

VI Thiết kế test case (Bonus)

1 Test case là gì?

Test case xác định một tình huống kiểm tra cụ thể: dữ liệu và trạng thái ban đầu, thao tác, kết quả mong đợi. "Kiểm tra đăng ký" là một mục tiêu quá rộng; "Khóa học còn một chỗ, gửi hai yêu cầu đồng thời, chỉ một đăng ký được xác nhận" mới là tình huống kiểm chứng được.

Thiết kế test case không đồng nghĩa đã chạy test. Expected result được viết trước theo yêu cầu. Actual result và trạng thái Pass/Fail chỉ được ghi sau khi có lần chạy và kết quả thật.

2 Các mức kiểm thử

Mức	**Kiểm tra gì và ví dụ**
Unit	Một đơn vị logic được cô lập, như Course.hasSeat hoặc công thức tính thời lượng sạc. Nhanh, nhưng không chứng minh transaction database thật hoạt động.
Integration	Các phần phối hợp, như repository với database hoặc adapter với nguồn sự kiện giả lập. Kiểm tra mapping, ràng buộc và rollback.
API/service	Luồng request qua xác thực, validation, nghiệp vụ và response. Có thể gồm nhiều thành phần và không cần mở trình duyệt.
System / end-to-end	Luồng sử dụng trên hệ thống kết hợp, từ màn hình tới kết quả. Bao phủ trải nghiệm nhưng thường chậm và khó định vị lỗi hơn.

Đây là cách phân biệt thực dụng cho bài nhóm; không cần tranh luận một tên công cụ thuộc duy nhất mức nào. Cần mô tả ranh giới đang kiểm tra và phần nào đã được thay thế bằng test double.

3 Stub, mock và fake

Stub trả dữ liệu đã chuẩn bị, như gateway trả một mức pin xác định. **Mock** thường dùng để kiểm tra tương tác kỳ vọng, như lệnh mở khóa chỉ được gửi một lần. **Fake** là hiện thực đơn giản vẫn có hành vi, như store trong bộ nhớ.

Tên gọi trong framework có thể khác, nên nói rõ cách test dùng nó. Fake không có isolation/ràng buộc giống database thật có thể khiến test concurrent đặt xe vượt qua mà lỗi vẫn tồn tại khi chạy thực tế.

4 Cấu trúc một test case

Trường	**Ý nghĩa**
ID và mục tiêu	Nhận diện tình huống và hành vi/rủi ro cần kiểm chứng.
Liên quan	UC, BR/FR/NFR và class/method mục tiêu.
Precondition	Trạng thái tài nguyên, quyền actor và môi trường ban đầu.
Test data	Input, ID, mức pin, thời gian, phiên bản và phản hồi nguồn ngoài.
Steps	Các bước có thể lặp lại; chỉ rõ nếu phải gửi đồng thời.
Expected result	Response cùng tác động dữ liệu và điều không được xảy ra.
Actual / evidence	Kết quả thực tế, phiên bản chạy, log hoặc báo cáo test; chưa chạy thì ghi rõ.

Arrange–Act–Assert là cách đọc ngắn: chuẩn bị dữ liệu, thực hiện thao tác, kiểm tra kết quả. Không chỉ assert mã HTTP mà bỏ trạng thái trong store. HTTP thành công nhưng tài nguyên bị gán hai lần vẫn là lỗi.

5 Equivalence partition và boundary value

Chia input thành các lớp có hành vi tương đương để không thử ngẫu nhiên quá nhiều. Ví dụ số chỗ còn lại: không còn chỗ, còn chỗ hợp lệ và dữ liệu sức chứa không hợp lệ.

Ưu tiên giá trị sát biên. Nếu một quy tắc cho phép mức pin từ 30% thì thử 29%, 30%, 31%. Nếu input cho phép số thập phân, chọn giá trị sát biên theo độ chính xác đã thống nhất, không mặc nhiên chỉ có số nguyên.

Với thời hạn expiresAt, thử ngay trước, đúng bằng và ngay sau thời hạn. Hợp đồng phải quyết định `now >= expiresAt` có bị coi là hết hạn không; người viết test không tự chọn thay nghiệp vụ.

6 Decision table và state-transition test

Có quyền	Còn hiệu lực	Xe hợp lệ	Kết quả mong đợi
Không	Bất kỳ	Bất kỳ	Từ chối; không thay đổi dữ liệu.
Có	Không	Bất kỳ	Không bàn giao; xử lý theo trạng thái hiện tại.
Có	Có	Không	Không bắt đầu chuyến đi; giữ kết quả kiểm tra thực tế.
Có	Có	Có	Chỉ bắt đầu khi nhận xác nhận bàn giao hợp lệ.

Decision table giúp thấy những tổ hợp bị bỏ sót. State-transition test kiểm tra cả chuyển hợp lệ và chuyển bị cấm: không phải chỉ chạy theo chuỗi thành công từ đầu tới cuối.

7 Kiểm thử cạnh tranh và lỗi giữa chừng

Khởi chạy nhiều tác vụ sau cùng một điểm đồng bộ, không dùng một vòng for gọi lần lượt rồi gọi đó là concurrent. Mỗi request có dữ liệu và actor phù hợp; khi thử cạnh tranh cùng xe, tránh một điều kiện khác đã làm tất cả request bị từ chối trước.

Sau khi hoàn tất, kiểm tra số lượt thành công, số active reservation, trạng thái xe và audit quan trọng. Với idempotency, thử lại cùng key; với một yêu cầu mới, dùng key khác. Đây là hai tình huống khác nhau.

Đưa lỗi vào giữa quá trình ghi để kiểm tra không có bản ghi mồ côi hoặc số lượng bị tăng sai. Nếu hệ thống đã gửi lệnh ra ngoài rồi mất phản hồi, phải kiểm tra phương án đọc lại/hòa giải chứ không chỉ assert có gọi rollback.

8 Kiểm thử NFR

Đo hiệu suất cần nêu tác vụ, điều kiện tải, thời gian chạy, phân vị hoặc thống kê sử dụng và tỷ lệ lỗi. Một request chạy nhanh trên máy cá nhân không chứng minh hệ thống chịu được giờ cao điểm.

Độ sẵn sàng theo tháng không thể được xác nhận chỉ bằng vài phút demo. Kiểm thử phục hồi có thể đo một phần mục tiêu nhưng không thay thế số liệu vận hành. Tương tự, có hash mật khẩu chưa chứng minh toàn bộ hệ thống an toàn.

9 Ví dụ test đã thiết kế, chưa phải kết quả chạy

ID	TC-COURSE-03
Mục tiêu	Hai người cùng đăng ký một chỗ còn lại.
Ban đầu	Cùng Course còn một chỗ; hai Student đang hoạt động và chưa đăng ký; store hỗ trợ điều kiện ghi/version như thiết kế.
Thao tác	Gửi hai yêu cầu với request ID khác nhau cùng lúc; đợi cả hai hoàn tất.
Mong đợi	Đúng một đăng ký Confirmed; số người không vượt capacity. Yêu cầu còn lại nhận Full hoặc Conflict theo hợp đồng đã chọn; không có bản ghi xác nhận thứ hai.
Kết quả thực tế	Chưa chạy; đây là ví dụ thiết kế test case.

VII Ví dụ hoàn chỉnh: đăng ký khóa học

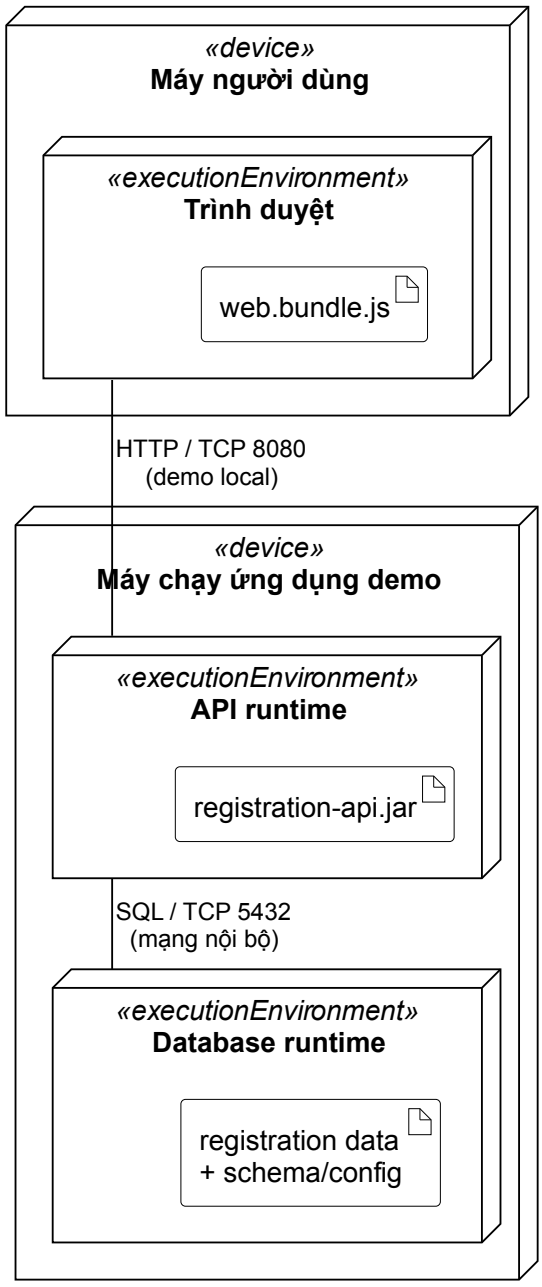
1 Bối cảnh và quy tắc của ví dụ

Đây là ví dụ ngoài SEMH. Một sinh viên đã xác thực chọn khóa học và yêu cầu đăng ký. Khóa học có sức chứa; tài khoản phải đang hoạt động. Mỗi sinh viên chỉ có một đăng ký được xác nhận cho cùng khóa học trong phạm vi ví dụ. Không có thanh toán, hủy đăng ký hay danh sách chờ.

Một request có requestId để khi mất phản hồi có thể gửi lại mà không tạo thêm đăng ký. Hai người cùng chọn chỗ cuối không được làm enrolledCount vượt capacity. Các quy tắc này là dữ liệu đầu vào của thiết kế, không phải điều tự suy ra từ hình.

CourseSession là buổi học thuộc nội dung Course. Ví dụ chọn composition cho nó vì đối tượng buổi học không có owner độc lập trong mô hình này. Chính sách lưu lịch sử ngoài phạm vi; không dùng điều này làm lý do tự xóa dữ liệu đăng ký.

2 Deployment View của ví dụ



Hình 26: *Nơi chạy các artifact của hệ thống đăng ký demo*

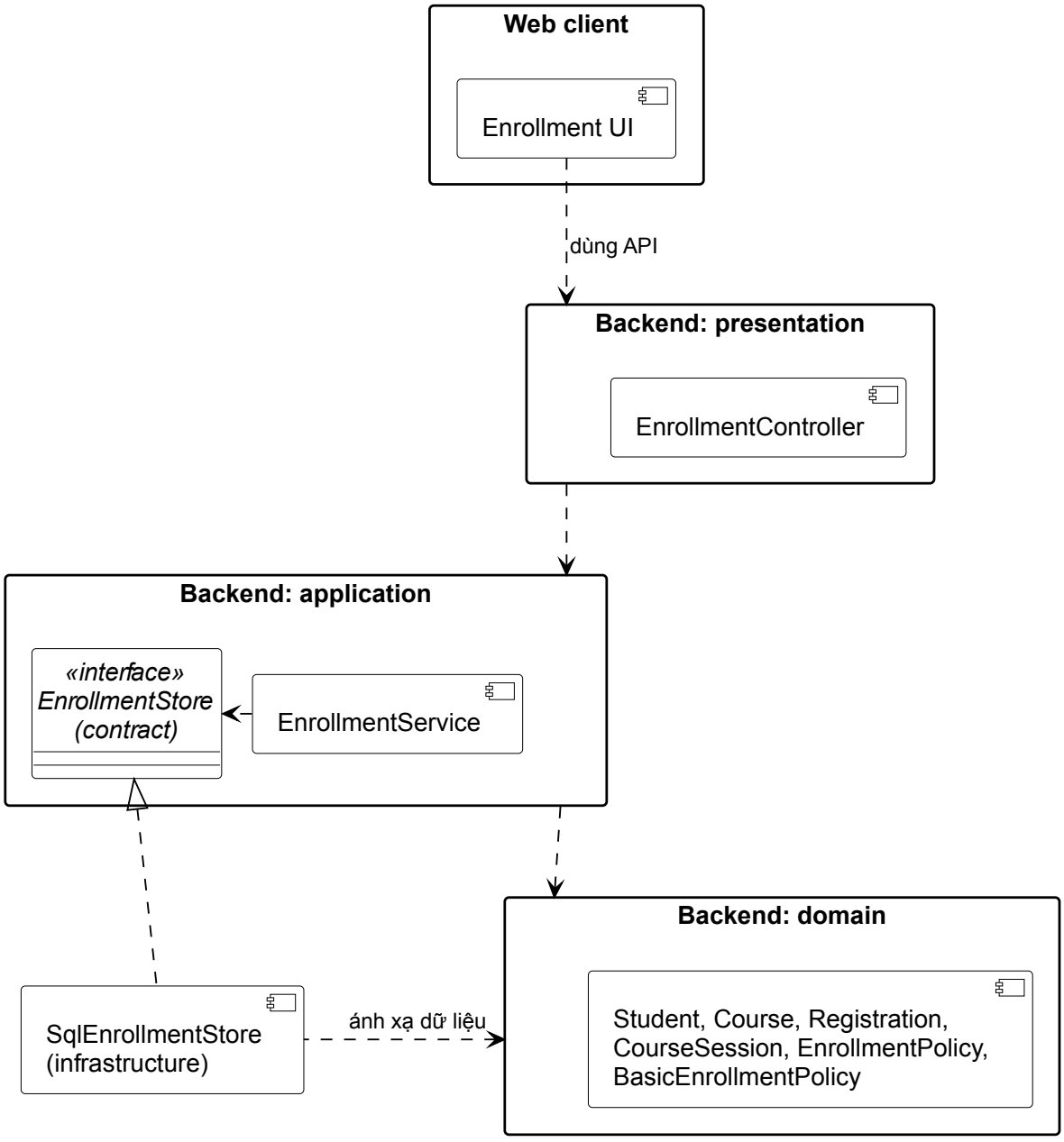
Cách đọc

Trình duyệt nạp web bundle và gửi request đến API runtime. Artifact backend chứa các phần nhận request, nghiệp vụ và truy cập dữ liệu. Runtime database giữ dữ liệu và cấu hình lưu trữ. Hai đường nối là đường giao tiếp, không phải main flow của việc đăng ký.

Giả định môi trường

HTTP 8080 và SQL 5432 là cấu hình minh họa của ví dụ, không phải port bắt buộc cho SEMH. Máy người dùng và máy demo có thể là hai thiết bị; khi chạy local, các runtime cũng có thể cùng nằm trên một máy. Khi lên môi trường công khai cần xem xét HTTPS, secret, giới hạn truy cập database và dữ liệu bền vững.

3 Development/Implementation View của ví dụ

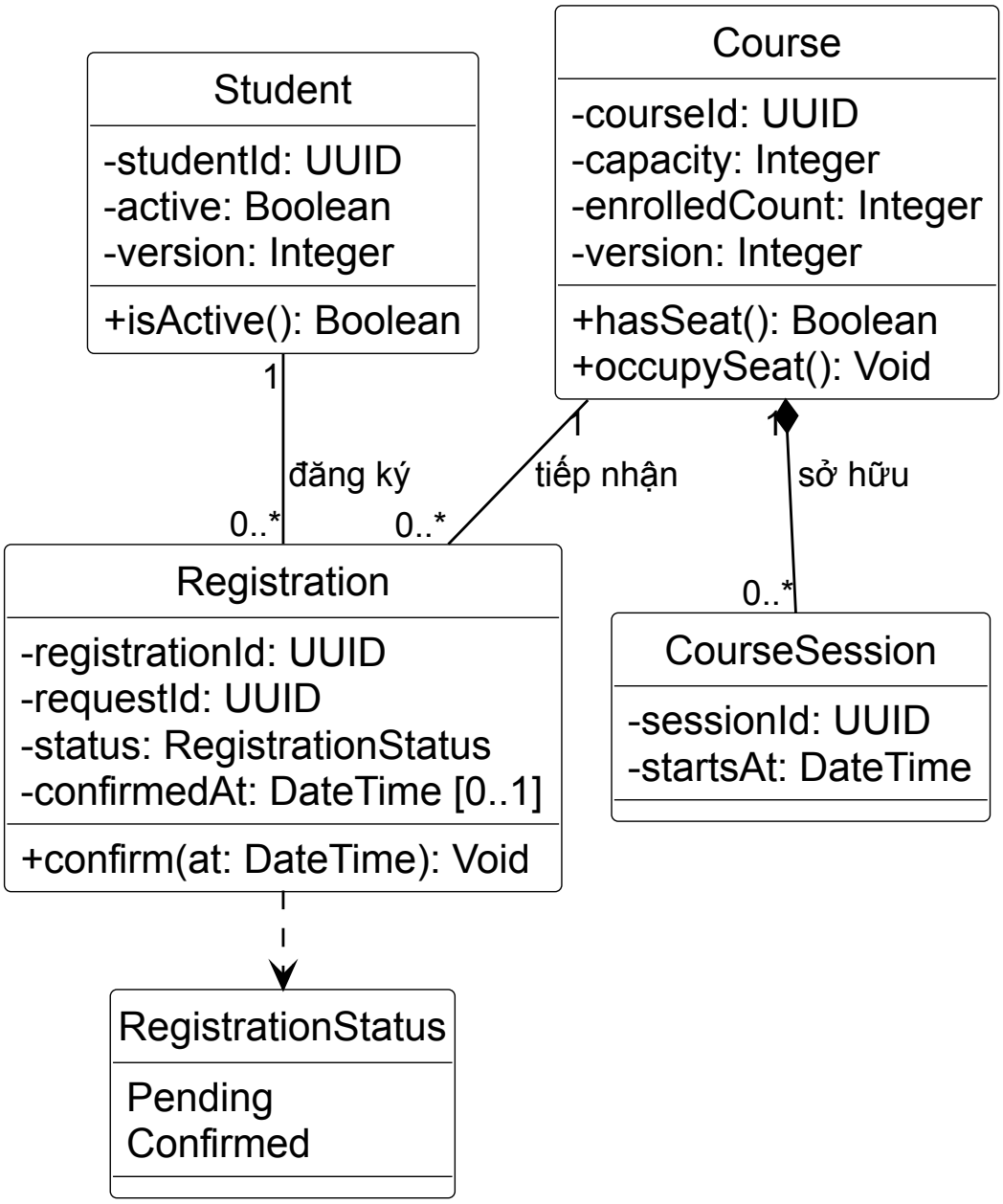


Hình 27: *Phụ thuộc giữa các phần mã nguồn; không phải các server riêng*

Web UI sử dụng API của controller. Controller gọi EnrollmentService. Service làm việc với các đối tượng miền và hợp đồng EnrollmentStore/EnrollmentPolicy. SqlEnrollmentStore là hiện thực lưu trữ; BasicEnrollmentPolicy là chính sách kiểm tra.

Mũi tên từ SqlEnrollmentStore về EnrollmentStore cho biết implementation đáp ứng contract. Ở runtime, service được truyền một object SqlEnrollmentStore rồi gọi nó. Không có request mạng từ interface đến SQL class như một server độc lập.

4 Class diagram miền

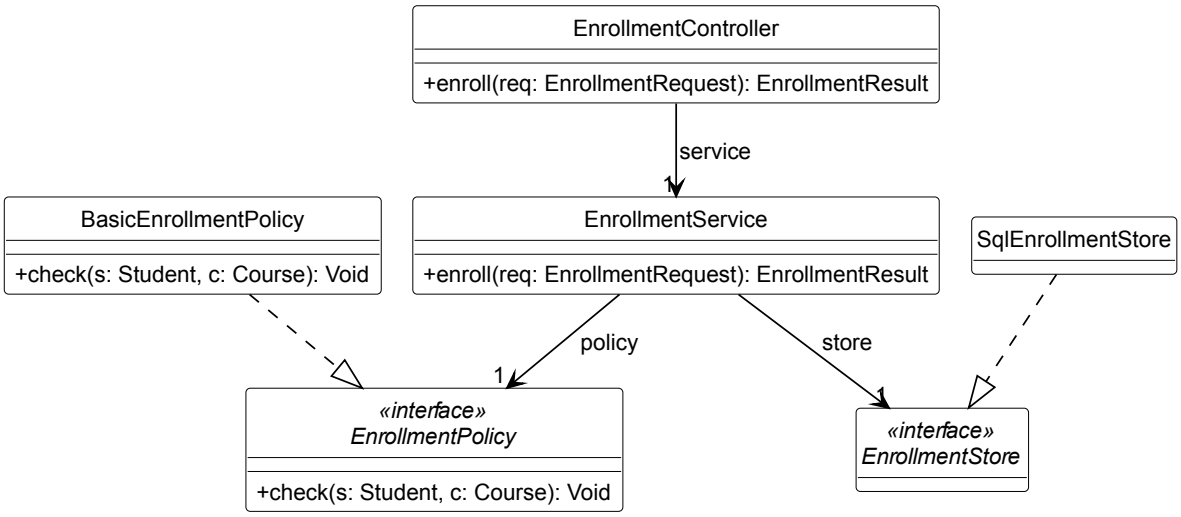


Hình 28: *Các đối tượng miền và quan hệ của ví dụ*

- Student liên hệ nhiều Registration; mỗi Registration ứng với đúng một Student và một Course.
- Course giữ capacity, enrolledCount và version. `0 <= enrolledCount <= capacity` là invariant.
- Registration.confirm chỉ thay đổi object trong bộ nhớ; muốn trở thành kết quả đã lưu còn phải commit thành công.
- CourseSession không có operation trên hình này nên không cần bịa thêm getter để lấp ngăn method.

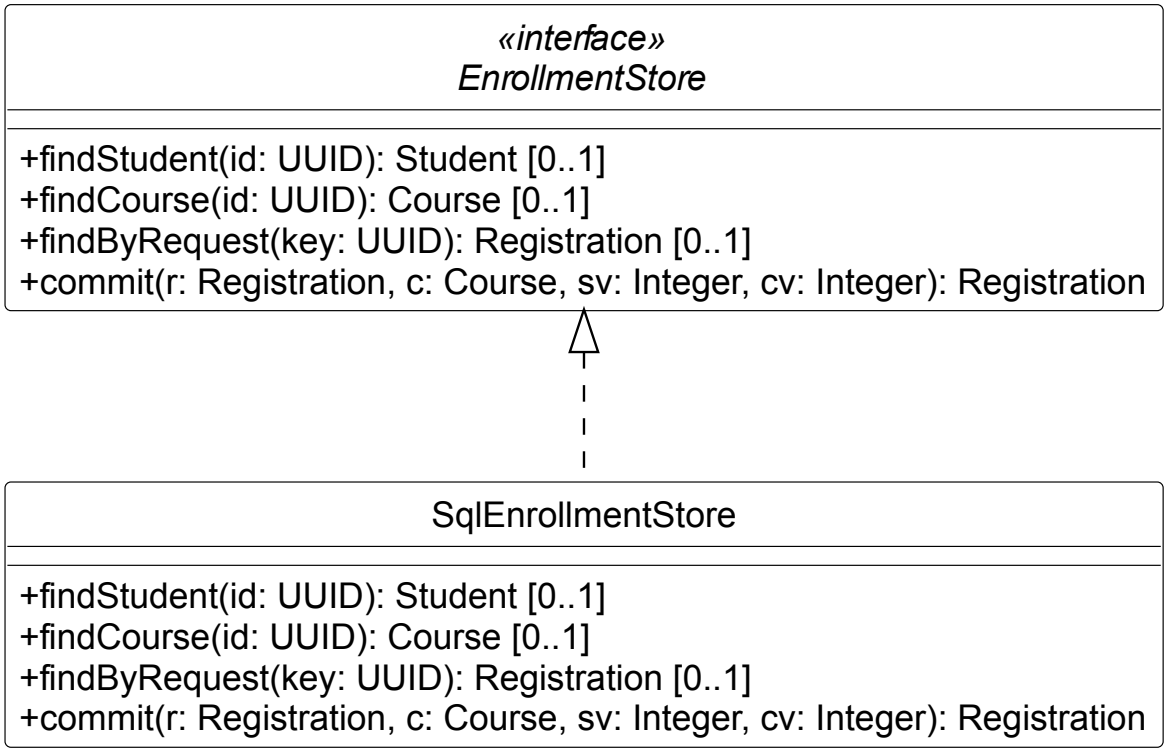
Hai giá trị RegistrationStatus chỉ là phạm vi ví dụ. Chúng không thay cho các trạng thái VehicleReservation của SEMH. Nhiều đăng ký lịch sử vẫn được biểu diễn 0..*; ràng buộc không trùng theo sinh viên và khóa học được kiểm tra riêng.

5 Class diagram điều phối và lưu trữ



Hình 29: *Các lớp điều phối, interface và implementation tương ứng*

Hình trên ẩn các operation lưu trữ để giữ cỡ chữ dễ đọc. Hình dưới mở đúng hai kiểu đó, không tạo thêm interface hoặc adapter khác.



Hình 30: *Chi tiết hợp đồng lưu trữ và class hiện thực*

Trong signature commit, `sv` là version Student đã kiểm tra, `cv` là version Course đã đọc. Dùng tên ngắn trên hình để dễ dàn trang; khi hiện thực nên đặt expectedStudentVersion và expectedCourseVersion rõ nghĩa. Registration đã chứa requestId nên không cần truyền key lặp lại vào commit.

Các kiểu dữ liệu truyền qua ranh giới

Kiểu	**Nội dung**
EnrollmentRequest	studentId lấy từ danh tính đã xác thực ở server; courseId do người dùng chọn; requestId để nhận diện lần gửi lặp.
EnrollmentResult	Mã kết quả; registrationId khi thành công; thông tin lỗi nghiệp vụ an toàn cho bên gọi. Không trả object lưu trữ nguyên vẹn ra UI.
UUID / DateTime	Kiểu định danh và mốc thời gian dùng chung trong ví dụ; DateTime được quy ước UTC khi trao đổi/lưu.

6 Hợp đồng các operation miền

ID	Owner / operation	Hợp đồng
M01	Student.isActive()	Trả true khi tài khoản hoạt động; không ghi dữ liệu, không thay đổi quyền. Dữ liệu tài khoản phải đã được nạp.
M02	Course.hasSeat()	Trả enrolledCount nhỏ hơn capacity. Không giữ chỗ và không bảo đảm chỗ còn tồn tại khi một request khác vừa commit.
M03	Course.occupySeat()	Chỉ tăng một đơn vị khi còn chỗ. Không còn chỗ thì báo lỗi miền và không thay đổi object. Chỉ sửa trong bộ nhớ, chưa là một giao dịch lưu.
M04	Registration confirm(at)	Chỉ từ Pending sang Confirmed; ghi confirmedAt bằng at hợp lệ. Trạng thái không phù hợp thì từ chối, giữ nguyên object. Không tự tăng số người của Course.

7 Hợp đồng điều phối và policy

ID	Owner / operation	Hợp đồng
M05	EnrollmentController enroll(req)	Nhận input, kiểm tra phiên và dữ liệu hình thức; gán studentId từ danh tính server; gọi service và ánh xạ kết quả sang response. Không tự tăng enrolledCount. Không có phiên thì không gọi thao tác nghiệp vụ.
M06	EnrollmentService enroll(req)	Điều phối kiểm tra request ID, tải dữ liệu, policy, thay đổi object và commit. Trả kết quả đã xác nhận hoặc lỗi có nghĩa; không trả thành công trước commit. Request lặp cùng nội dung trả kết quả cũ.
M07	EnrollmentPolicy check(s,c)	Contract: với Student/Course đã nạp, cho đi tiếp nếu tài khoản hoạt động và còn chỗ; nếu không thì báo Inactive hoặc Full. Không ghi dữ liệu.

ID	Owner / operation	Hợp đồng
M08	BasicEnrollmentPolicy check(s,c)	Hiện thực M07 bằng s.isActive() và c.hasSeat(). Không kiểm tra quyền HTTP, không commit; kết quả chỉ phản ánh trạng thái đã đọc, chưa giải quyết cạnh tranh.

8 Hợp đồng của interface lưu trữ

ID	Operation của EnrollmentStore	Hợp đồng
M09	findStudent(id)	Trả Student cùng version hoặc không tìm thấy. Chỉ đọc; lỗi lưu trữ khác với kết quả không tồn tại.
M10	findCourse(id)	Trả Course cùng capacity, enrolledCount và version, hoặc không tìm thấy. Không tăng số người và không tạo đăng ký.
M11	findByRequest(key)	Tra cứu đăng ký đã commit theo requestId. Service phải đối chiếu owner và course trước khi trả lại; không để key dùng lại sai nội dung gây lộ dữ liệu hoặc tạo tác động khác.
M12	commit(r,c,sv,cv)	Trong một transaction: bảo đảm version Student/Course vẫn khớp sv/cv; không trùng student–course hoặc requestId; ghi đăng ký Confirmed và số lượng mới; tăng version Course. Thất bại phải không ghi một phần. Request trùng đã commit được phân biệt để đọc lại kết quả.

9 Hợp đồng implementation và bao phủ toàn bộ method

SqlEnrollmentStore hiện thực đủ bốn operation M09–M12. Không bỏ phần mô tả implementation chỉ vì đã có interface; cần bổ sung cơ chế và lỗi đặc thù:

ID	Operation của SqlEnrollmentStore	Bổ sung cho contract interface
M13	findStudent(id)	Ánh xạ dòng tài khoản sang Student; giữ đúng version; lỗi kết nối được báo là lỗi hạ tầng, không biến thành "không có sinh viên".
M14	findCourse(id)	Đọc đúng enrolledCount/capacity/version từ cùng trạng thái lưu trữ thích hợp. Không tự chọn khóa phân tán ngoài hợp đồng; bảo vệ cạnh tranh nằm ở commit.
M15	findByRequest(key)	Query theo key duy nhất. Không có bản ghi là optional rỗng; có bản ghi trả đúng owner/course/status để M06 kiểm tra replay.
M16	commit(r,c,sv,cv)	Hiện thực M12 bằng kiểm tra version có khóa hoặc điều kiện cập nhật, ràng buộc duy nhất và transaction SQL. Nếu xác nhận rollback, giữ nguyên dữ liệu cũ; nếu kết quả commit chưa rõ, báo cần đọc lại theo requestId, không khẳng định chắc chắn chưa có đăng ký.

Ba hình class diagram thể hiện tổng cộng **16 operation phân biệt theo owner**, tương ứng M01–M16; hai hình điều phối và lưu trữ là các góc nhìn bổ sung của cùng thiết kế. Constructor và getter không xuất hiện không được tính là đã mô tả trên hình. Nếu sau này thêm operation lên sơ đồ, phải bổ sung hợp đồng của nó; không dùng số 16 như một chỉ tiêu cố định.

10 Đọc đường xử lý của EnrollmentService

1. Kiểm tra request hợp lệ và studentId đã được xác thực từ server.
2. Tra theo requestId. Nếu đã có kết quả cùng Student/Course, trả lại kết quả đó; nếu khác nội dung, từ chối key dùng sai.
3. Tải Student và Course; không tìm thấy thì kết thúc mà không ghi.
4. Lưu version gốc sv/cv; gọi policy kiểm tra các điều kiện nghiệp vụ.
5. Gọi Course.occupySeat(), tạo Registration Pending và confirm bằng thời điểm hợp lệ.
6. Gọi commit để ghi tất cả. Chỉ sau thành công mới trả Confirmed.
7. Nếu conflict do tác vụ cạnh tranh, không tự dùng lại object cũ để ghi. Trả Conflict hoặc đọc lại và đánh giá lại theo chính sách retry hữu hạn.

Đăng ký cùng course của cùng Student cũng cần ràng buộc duy nhất. Version Course giúp tránh ghi đè số lượng nhưng không tự thay thế mọi ràng buộc nghiệp vụ.

11 Hai request cạnh tranh một chỗ

Giả sử capacity là 10, enrolledCount là 9, version Course là 7. Hai request cùng đọc bản 7 và cùng tính số lượng mới là 10. Chỉ transaction đầu ghi thành công với expected version 7, rồi tăng version thành 8. Transaction còn lại gặp version khác nên không thể ghi tiếp theo dữ liệu cũ.

Không được để cả hai cùng lưu Registration Confirmed trong khi enrolledCount chỉ tăng một lần. Transaction phải bao trùm cả bản ghi Registration, cập nhật Course và các ràng buộc liên quan.

12 Test case rút ra trực tiếp từ thiết kế

Tình huống	**Input / trạng thái**	**Expected result**
Thành công	Student active, còn chỗ, request ID mới	Một đăng ký Confirmed; enrolledCount tăng một; đúng thời điểm xác nhận.
Hết chỗ	enrolledCount bằng capacity	Full; không tăng số lượng, không ghi Registration mới.
Gửi lặp	Cùng request ID, Student và Course	Trả đăng ký cũ; không có tác động thứ hai.
Sai key	Cùng request ID nhưng đổi Course/owner	Từ chối nội dung không khớp; không lộ bản ghi của người khác.
Cạnh tranh	Hai Student cùng chọn chỗ cuối	Chỉ một xác nhận; request còn lại Full hoặc Conflict theo hợp đồng.
Lỗi ghi	Giao dịch lỗi trước commit và rollback được xác nhận	Không có thay đổi một phần; không trả Confirmed.
Mất phản hồi	Commit có thể đã thành công nhưng mất kết nối	Đọc lại cùng request ID; không mặc định tạo yêu cầu mới.

Tất cả dòng trên là **test được thiết kế**, chưa phải bằng chứng đã chạy một ứng dụng. Ví dụ cho thấy các view và method descriptions nối với nhau thế nào, không áp đặt stack của nó lên hệ thống SEMH.

VIII Áp dụng cho SEMH và tự luyện

1 Đầu vào hiện hành của nhóm

Baseline hiện tại là báo cáo Phase 2 tích hợp: **23 UC, bảy phân hệ và bốn vai trò người dùng**. GEN có ba UC quản trị dùng chung; SUB-OPS có thêm UC-OPS-04/05. Không quay về phạm vi 18 UC cũ hoặc bỏ Admin khi sang thiết kế.

Trong tài liệu này, những class/method mới ở phần gợi ý là **ứng viên thiết kế**, không phải quyết định công nghệ hoặc giao diện đã được nhóm khóa. Tên đối tượng và trạng thái có sẵn phải giữ theo quy chuẩn Phase 2. Các quyết định mới cần thống nhất trước khi điền vào báo cáo chính.

2 Nhìn hệ thống bằng hai view kiến trúc

Deployment View

Xác định nơi chạy website, backend, nơi lưu dữ liệu, nguồn trạng thái giả lập/thực và dịch vụ bản đồ. Worker mô phỏng có thể ở cùng backend hoặc là tiến trình riêng tùy phương án; cần nói rõ, không tự suy ra một server cho mỗi phân hệ.

Development/Implementation View

Giữ các ranh giới HUB, RES, TRIP, PARK, CHG, OPS, SIM cùng chức năng dùng chung GEN. Một phương án học dễ theo là backend chia module, dùng interface ở các ranh giới dữ liệu/nguồn ngoài. Đây là phương án để cân nhắc, không phải stack đã chọn.

Điều cần thống nhất trước khi bảy người cùng thiết kế

Kiểu ID, kiểu thời gian, đơn vị pin/công suất, kết quả lỗi, nguồn danh tính, store contract và cách mở transaction. Cùng tên Service trong sequence không bắt buộc phải thành một ứng dụng deploy riêng.

3 Gợi ý trách nhiệm theo phân hệ

Phạm vi	Đối tượng và vấn đề cần thiết kế
GEN	Account, AccessAssignment, AccessService; quản trị tài khoản và phạm vi; không để Admin mặc nhiên có quyền vận hành.
HUB	MobilityHub và dữ liệu tài nguyên; truy vấn theo vị trí, thời điểm cập nhật, fallback và đánh dấu chưa xác định. Không giữ tài nguyên chỉ vì xem thông tin.

Phạm vi	Đối tượng và vấn đề cần thiết kế
RES	VehicleReservation; một lượt đặt hiệu lực trên mỗi sinh viên và mỗi xe; xác nhận, hủy, hết hạn và xử lý gửi lặp trong kiểm tra cạnh tranh.
TRIP	Trip, SharedVehicle; bàn giao vật lý, trả xe, chốt số liệu; kết thúc Trip không tự bảo đảm SharedVehicle.Available.
PARK	ParkingReservation, ParkingSpace, PrivateVehicle; giữ chỗ, check-in/out, hết hạn và trạng thái lỗi có thể tới cùng lúc.
CHG	ChargingRequest, ChargingSession, ChargingPoint; ưu tiên, thời lượng, tương thích cổng và tổng công suất; xem lịch khác điều chỉnh lịch.
OPS	Incident, RedistributionPlan, ServiceStatus; quyền kỹ thuật/vận hành, đóng sự cố, điều chuyển và bật/tắt tài nguyên có ràng buộc.
SIM	SimulationScenario, SimulationRun, Recommendation; đóng băng đầu vào lần chạy, snapshot, tiến độ, kết quả và khuyến nghị không ghi vào mạng vận hành.

Một class dùng chung như SharedVehicle phải có định nghĩa chuẩn. RES dùng nó không có nghĩa tạo thêm một class SharedVehicle khác với bộ trạng thái riêng. Cũng không dùng composition Hub–SharedVehicle chỉ vì xe hiện đang được gán vào Hub; xe còn có thể được điều chuyển.

4 Lifeline của Phase 2 chuyển sang thiết kế như thế nào?

Tên hiện có	Ý nghĩa khi sang Phase 3
ReservationUI, HubUI	Boundary phía giao diện; có thể là trang/component web. Không phải bắt buộc tạo entity lưu trong database.
ReservationService	Trách nhiệm application cho nghiệp vụ đặt/hủy; tách thêm class chỉ khi có lý do.
SEMHDataStore	Trừu tượng lưu trữ nội bộ trong mô hình trước; nay cần chốt interface, ownership dữ liệu và cách hiện thực. Không có nghĩa chỉ là dữ liệu giả.
StateDataGateway	Tên chuẩn của nguồn trạng thái ngoài, tương ứng ACT-DAT/EXT-STATE. Phía nội bộ có thể cần adapter nhận, kiểm tra và ánh xạ sự kiện.
MapService	Nguồn bản đồ/định vị ngoài theo ACT-MAP/EXT-MAP. Không đổi thành một database nội bộ chỉ để dễ vẽ.

Nếu tách một lifeline trừu tượng thành nhiều lớp cụ thể, cần ghi cách ánh xạ và kiểm tra lại các thông điệp liên quan. Không đổi tên ở class diagram mà để sequence và mô tả method trỏ tới thành phần đã không còn ý nghĩa.

5 Ví dụ suy nghĩ thiết kế cho UC-RES-01

Một ứng viên application method là `reserveVehicle(context, vehicleId, key)` của ReservationService. Context đến từ danh tính/phiên đã xác thực; không lấy studentId tùy ý do UI gửi rồi coi là người đang đăng nhập.

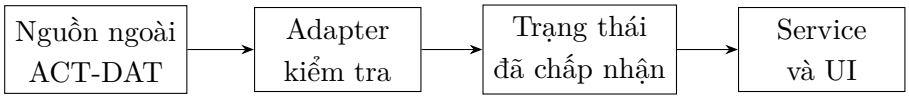
Các điều kiện cần được bảo vệ cùng một quyết định cấp phát gồm: sinh viên không có lượt Pending/Confirmed khác, xe không bị giữ, mức pin đạt ngưỡng, Hub và xe đều được phục vụ, không có lỗi hoặc ràng buộc khác ngăn đặt. Chỉ kiểm tra VehicleStatus.Available là chưa đủ theo baseline.

Khi xác nhận thành công, VehicleReservation.Confirmed và việc giữ SharedVehicle phải nhất quán. Trường hợp hai request cạnh tranh không được cả hai cùng xác nhận. Request gửi lặp sau mất phản hồi cần tra kết quả theo key, không tự tạo lần giữ thứ hai.

Hết hạn là trách nhiệm nền NIFR-RES-01, không phải actor "Timer" bên ngoài. Method xử lý phải kiểm tra lượt vẫn Confirmed và chưa được nhận. Việc trả xe về Available còn phụ thuộc điều kiện phục vụ; không gán Available một cách vô điều kiện.

6 IoT giả lập vẫn cần được thiết kế đầy đủ

Luồng cần nhìn được là: **nguồn ngoài** gửi sự kiện, **adapter/ingestion nội bộ** kiểm tra và chuyển đổi, **store** giữ trạng thái đã chấp nhận, **service/UI** sử dụng dữ liệu có nguồn và thời điểm rõ ràng.



Hình 31: *Nguồn dữ liệu ngoài khác với nơi lưu dữ liệu của hệ thống*

Sự kiện nên xác định được tài nguyên, loại trạng thái, đơn vị, thời gian và key/version phù hợp với phương án. Cần xử lý sự kiện trùng, tới muộn, thiếu trường và mất kết nối. Hiển thị dữ liệu cũ có cảnh báo không có nghĩa dùng nó để cấp phát một tài nguyên mà chưa kiểm tra lại.

Theo hướng dẫn của cô, Azure Stream Analytics là công cụ tham khảo cho luồng real-time; không phải mọi nhóm bắt buộc thêm Azure IoT Hub, broker và nhiều dịch vụ cloud. Dữ liệu giả lập vẫn là actor/nguồn ngoài; bên trong SEMH vẫn cần lớp nhận và quản lý nó. Thêm component chỉ khi nhóm có trách nhiệm thật cần hiện thực.

7 Thiết kế phân hệ mô phỏng (SUB-SIM)

Kịch bản khác lần chạy

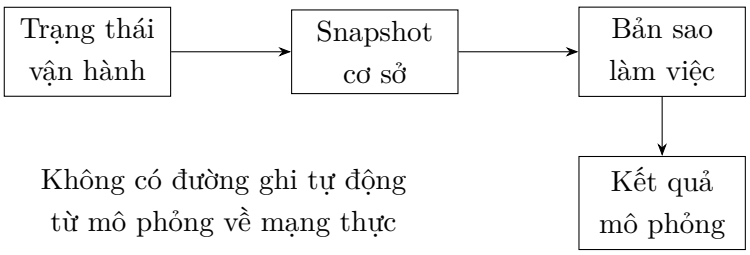
SimulationScenario giữ cấu hình dùng để thử nghiệm. SimulationRun là một lần thực thi với đầu vào, trạng thái và kết quả cụ thể. Chạy lại cùng kịch bản có thể tạo run mới để giữ lịch sử; không ghi đè kết quả cũ chỉ vì tên scenario không đổi.

Tham số cần có kiểu, đơn vị và phạm vi

Ví dụ tăng nhu cầu 30% cần biết tăng so với cơ sở nào, trong khoảng thời gian nào và tại các Hub nào. "Thời gian mô phỏng 60 phút" khác số giây chương trình mất để tính. Random seed có thể hỗ trợ tái lập nếu mô hình dùng ngẫu nhiên; đây là lựa chọn kỹ thuật cần thống nhất, không phải tính năng đã được thêm vào baseline.

Snapshot và dữ liệu làm việc

Snapshot là dữ liệu cơ sở có mốc/phiên bản. Run dùng bản sao làm việc để tính trạng thái giả định. Trong lúc mạng thực thay đổi, không tự làm snapshot trôi theo mà không có định nghĩa, vì kết quả sẽ khó đối chiếu.



Hình 32: *Tính kết quả ở vùng riêng không đồng nghĩa điều khiển thiết bị thật*

Các trách nhiệm method có thể cần

Tạo/cập nhật scenario, kiểm tra cấu hình, bắt đầu run, lấy tiến độ, đọc kết quả và phân tích khuyến nghị. Có thể phân công cho SimulationService và các đối tượng/engine hỗ trợ; không cần biến mỗi trách nhiệm thành một server.

Application method bắt đầu run phải chỉ rõ khi nào ghi Running, lúc nào xác nhận Completed/Failed, dữ liệu nào được lưu khi lỗi và cách đọc lại sau khi UI đóng. Giữ đúng bốn trạng thái Draft, Running, Completed, Failed hiện có; không tự thêm Cancelled vì một khung cũ có chữ "hủy".

Khuyến nghị không phải lệnh đã được áp dụng

Không có khuyến nghị có thể là kết quả hợp lệ. Phân tích chưa hoàn thành lại là tình huống khác. Chọn khuyến nghị để lập RedistributionPlan phải đi qua quyền và quy trình của OPS; không tạo method trong engine để tự thay Hub quản lý của xe thật.

Test nên ưu tiên

Sai đơn vị/phạm vi tham số; khôi phục tiến độ khi mở lại UI; kết quả rỗng hợp lệ; trạng thái Failed; cùng đầu vào/seed khi cần tái lập; và cách ly kho vận hành. Với test isolation, dùng fixture được kiểm soát hoặc theo dõi nguồn thao tác ghi. Không khẳng định toàn mạng không đổi trong khi người dùng thực vẫn đang hoạt động.

8 Cách chia việc mà không nhân bản thiết kế

Các thành viên tiếp tục phụ trách phân hệ đang làm. Kiến trúc tổng thể và hợp đồng dùng chung cần được thống nhất cùng nhau trước. Người phụ trách class chuẩn giữ trách nhiệm thay đổi định nghĩa; thành viên khác dẫn chiếu thay vì chép thành phiên bản mới.

Class diagram có thể tách theo phân hệ để dễ đọc, nhưng các quan hệ liên phân hệ phải ăn khớp. Method description đặt tại owner của method; từng UC ghi rõ nó dùng method nào. Với GEN và hai UC-OPS bổ sung, phải có người tiếp tục trách nhiệm khi phân công Phase 3.

9 Bài tập tự luyện và gợi ý đáp án

Bài 1: phân biệt view

"SimulationService chạy trên Node A" và "SimulationService phụ thuộc SnapshotStore" thuộc cùng một view không? *Gợi ý:* Câu đầu nói nơi thực thi; câu sau nói ranh giới/phụ thuộc mã nguồn. Nếu SnapshotStore có runtime riêng, mô tả thêm artifact/node tương ứng ở Deployment View, không đồng nhất class với server.

Bài 2: đọc multiplicity

SharedVehicle liên hệ 0..* VehicleReservation trong lịch sử. Làm sao giữ quy tắc tối đa một lượt Pending/Confirmed? *Gợi ý:* Giữ multiplicity lịch sử, thêm invariant theo trạng thái và cơ chế cạnh tranh tại nơi ghi. Không đổi 0..* thành 0..1 trên toàn lịch sử.

Bài 3: chọn quan hệ

ReservationService nhận MapResult làm tham số trong một thao tác nhưng không giữ nó. Có cần composition không? *Gợi ý:* Không có bằng chứng về sở hữu vòng đời. Dependency có thể đủ; nếu giữ tham chiếu cấu trúc lâu dài thì xem xét association.

Bài 4: phát hiện sai chiều mũi tên

Một hình vẽ EnrollmentStore với tam giác rỗng trỏ vào SqlEnrollmentStore. Đúng chưa? *Gợi ý:* Với realization, tam giác rỗng phải ở đầu interface EnrollmentStore; đường nối nét đứt xuất phát từ implementation.

Bài 5: method có hợp đồng chưa đủ

"completeTrip(id): Void – đổi trạng thái thành Completed". Còn thiếu gì? *Gợi ý:* Quyền và owner, trạng thái hiện tại, bằng chứng trả tại Hub hợp lệ/khóa an toàn, thời gian và số liệu chốt, tác động xe, transaction, lỗi/mất phản hồi và thử lại. Không mặc định xe trở thành Available.

Bài 6: transaction và thế giới ngoài

API đã gửi lệnh mở khóa nhưng không nhận được phản hồi. Có thể chỉ rollback database rồi coi xe chắc chắn còn khóa không? *Gợi ý:* Không. Cần đọc lại hoặc hòa giải trạng thái; cơ chế thiết bị ngoài không rollback tự động cùng SQL.

Bài 7: kiểm chứng phần bonus

Đã viết 20 test case trên giấy thì có được ghi 20/20 Pass không? *Gợi ý:* Chưa. Phải chạy, giữ dữ liệu/môi trường và evidence. Thiết kế test và thực thi test là hai bước khác nhau.

Bài 8: độ chính xác của tên gọi

Nhóm không dùng database thật mà lưu dữ liệu trong bộ nhớ. Có cần xóa SEMHDataStore khỏi thiết kế không? *Gợi ý:* Không, nếu nó vẫn là trách nhiệm lưu/truy vấn nội bộ. Nhưng phải nêu mất dữ liệu khi restart, cơ chế cạnh tranh và giới hạn so với NFR đã cam kết; không tự tuyên bố đạt lưu audit 90 ngày.

10 Tài liệu đối chiếu

1. **Đề bài CO3001**: `BTL_SoftwareEngineering_HK261_v1.pdf`, mục Project Submission Detail, Submission #3; phân biệt nội dung bắt buộc và test case bonus.
2. **OMG UML 2.5.1**: trang đặc tả chính thức; đối chiếu phần Properties/Associations, Classes, Components và Deployments. Chú ý ngữ nghĩa composition, dependency và realization.
3. **PlantUML**: Class diagram, Deployment diagram, Component diagram; tài liệu cú pháp công cụ, không thay thế định nghĩa ngữ nghĩa UML.
4. **Microsoft Architecture guidance**: Common web application architectures; đối chiếu layers, monolith và chiều phụ thuộc. Tài liệu dùng .NET để minh họa, không bắt nhóm chọn .NET hoặc Azure.
5. **Baseline nhóm**: báo cáo Phase 2 tích hợp và tệp quy chuẩn thuật ngữ Phase 2; giữ tên actor, đối tượng và trạng thái đã chốt. Đường dẫn nguồn nằm trong README của tài liệu.